KidCare

Child Care Management System

KẾ HOẠCH DỰ ÁN — BẢN DỄ HIỂU

Dành cho nhóm 5 sinh viên lần đầu làm dự án phần mềm · Mô hình Incremental

Dành cho ai	Nhóm sinh viên mới bắt đầu học Công nghệ phần mềm. Không cần biết trước thuật ngữ — phần 2 giải thích tất cả.
Cách dùng	Đọc phần 1–3 trước khi làm gì khác. Sau đó làm theo thứ tự từng phần; mỗi phần có **checklist** ☐ để biết khi nào xong.
Giả định	12 tuần, 5 người, không có khách hàng thật (giảng viên đóng vai khách hàng).
Tài liệu đi kèm	KidCare_Ke_Hoach_Chi_Tiet.pdf (bản kỹ thuật đầy đủ hơn) và README.md.

Mục lục

1. Bắt đầu từ đây . . . 3
2. Từ điển thuật ngữ . . . 4
3. Nhóm sẽ làm việc như thế nào? . . . 7
4. Tuần 1 — Chuẩn bị nền tảng . . . 9
5. Increment 0 — Làm móng (Tuần 1–2) . . . 10
6. Increment 1 — Đăng nhập và phân quyền (Tuần 3–4) . . . 13
7. Increment 2 — Quản lý trẻ, lớp, phụ huynh, giáo viên (Tuần 5–6) . . . 15
8. Increment 3 — Điểm danh (Tuần 7–8) . . . 17
9. Increment 4 — Sức khỏe (Tuần 9) . . . 19
10. Increment 5 — Hoạt động và Thông báo (Tuần 10) . . . 21
11. Hoàn thiện và bảo vệ (Tuần 11–12) . . . 23
12. Quy tắc làm việc nhóm . . . 25
13. Nếu gặp khó khăn . . . 27
Phụ lục A — Cấu trúc thư mục . . . 28
Phụ lục B — Mẫu tài liệu . . . 28
Phụ lục C — Checklist nộp bài . . . 29

1. Bắt đầu từ đây

1.1. KidCare là gì?

Hãy tưởng tượng một trường mầm non có vài trăm bé. Mỗi sáng, cô giáo ghi tên các bé đến lớp vào sổ giấy. Hồ sơ sức khỏe nằm trong tủ của nhân viên y tế. Ảnh hoạt động gửi qua nhóm chat. Phụ huynh muốn biết con đã đến trường chưa thì phải gọi điện hỏi.

KidCare là một website gom tất cả những việc đó vào một chỗ:

- Cô giáo điểm danh bằng vài cú bấm; phụ huynh mở điện thoại là thấy con đã đến lớp lúc mấy giờ.
- Nhân viên y tế ghi chiều cao, cân nặng, dị ứng, tiêm chủng của từng bé.
- Cô giáo đăng hoạt động trong ngày kèm ảnh; phụ huynh nhận thông báo.
- Quản lý trường xem danh sách lớp, trẻ, giáo viên ở một nơi.

1.2. Nhóm bạn sẽ làm gì trong 12 tuần?

Việc	Nghĩa là gì	Chiếm khoảng
Viết tài liệu	Mô tả phần mềm cần làm gì (yêu cầu), thiết kế nó ra sao (sơ đồ, bảng dữ liệu, API)	35%
Lập trình	Viết phần **Backend** (xử lý dữ liệu, chạy ở máy chủ) và **Frontend** (giao diện người dùng thấy)	40%
Kiểm thử	Kiểm tra phần mềm chạy đúng, không có lỗi nghiêm trọng	15%
Triển khai và bảo vệ	Đưa phần mềm lên chạy được, chuẩn bị báo cáo, demo	10%

1.3. Ba lời khuyên cho người mới

1. **Làm nhỏ, chạy được trước.** Đừng viết cả hệ thống rồi mới chạy thử. Làm xong một phần nhỏ, chạy được, rồi mới làm phần tiếp theo. Đó chính là ý nghĩa của mô hình Incremental.
2. **Viết ra trước khi làm.** Nhóm 5 người mà mỗi người hiểu một kiểu thì sẽ code ra 5 kiểu khác nhau. Tài liệu ngắn gọn giúp cả nhóm hiểu giống nhau.
3. **Hỏi sớm, đừng giấu lỗi.** Kẹt quá 2 giờ thì hỏi người khác trong nhóm. Trễ tiến độ mà nói sớm thì còn cắt bớt phạm vi được; nói muộn thì hết cách.

2. Từ điển thuật ngữ

Bạn sẽ gặp các từ này liên tục. Không cần thuộc lòng, cứ quay lại bảng này khi quên.

2.1. Về yêu cầu

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
FR **Yêu cầu chức năng**	Một điều phần mềm **phải làm được**. Mỗi FR có một mã để dễ nhắc tới.	FR-ATT-01: Giáo viên điểm danh được trẻ
NFR **Yêu cầu phi chức năng**	Phần mềm phải làm **tốt đến mức nào**: nhanh, an toàn, ổn định. Nên có con số để kiểm tra được.	Mật khẩu phải được mã hóa; API trả lời dưới 500 ms
Actor	Một **loại** người dùng (không phải một người cụ thể).	Phụ huynh, Giáo viên, Nhân viên y tế, Quản lý, Admin
Stakeholder	Người có liên quan hoặc bị ảnh hưởng bởi phần mềm (rộng hơn actor).	Phụ huynh, giáo viên, ban giám hiệu, và cả trẻ em
Use case (UC)	Một việc cụ thể actor muốn làm, **kể từng bước**.	UC-ATT-02: Giáo viên check-in cho trẻ
MoSCoW	Cách xếp độ ưu tiên: **M**ust (phải có), **S**hould (nên có), **C**ould (có thì tốt), **W**on't (lần này không làm).	Điểm danh = Must; Reporting = Could

2.2. Về quy trình

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
Incremental	Làm và giao phần mềm **từng phần**. Mỗi phần xong là chạy được, không đợi làm hết.	Phần 1: đăng nhập. Phần 2: quản lý trẻ. Phần 3: điểm danh...
Increment	Một 'phần' như vậy.	Increment 3 = tính năng Điểm danh
Module	Một khối chức năng gọn của hệ thống.	Module Điểm danh, module Sức khỏe
Retrospective	Buổi họp nhìn lại sau mỗi phần: cái gì tốt, cái gì chưa, lần sau đổi gì.	Họp 30 phút sau mỗi Increment
Definition of Done	Danh sách điều kiện để được gọi là 'xong' một việc.	Có test, đã được review, chạy được
Bảng truy vết	Bảng nối **yêu cầu → use case → API → test** để chứng minh không sót gì.	FR-ATT-01 → UC-ATT-01 → POST /attendance/check-in → TC-ATT-01

2.3. Về thiết kế

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
UML	Bộ ký hiệu chuẩn để vẽ sơ đồ phần mềm. Gồm 4 loại sơ đồ bên dưới.	—
Use Case Diagram	Vẽ **ai** (actor) làm được **việc gì** (use case).	Hình người 'Giáo viên' nối với bầu dục 'Điểm danh'
Activity Diagram	Lưu đồ các **bước** của một quy trình, có rẽ nhánh.	Các bước điểm danh buổi sáng
Sequence Diagram	Ai **gọi** ai theo thứ tự thời gian.	Giáo viên → Giao diện → Backend → Database
Class Diagram	Các 'đối tượng' trong hệ thống và quan hệ giữa chúng.	Child, Parent, Class, Attendance
ERD	Sơ đồ các **bảng dữ liệu** và quan hệ giữa chúng.	Bảng children nối với bảng classes
Wireframe	Bản phác giao diện, vẽ tay hoặc dùng Figma.	Màn hình điểm danh của giáo viên
ADR	Ghi lại một quyết định kỹ thuật và **lý do** chọn.	Vì sao chọn Prisma thay vì TypeORM

2.4. Về kỹ thuật

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
Frontend	Phần giao diện người dùng nhìn và bấm (chạy trên trình duyệt).	React
Backend	Phần xử lý phía sau, đọc/ghi dữ liệu, kiểm tra quyền (chạy trên máy chủ).	NestJS
Database (CSDL)	Nơi lưu dữ liệu lâu dài, gồm nhiều **bảng**.	PostgreSQL
API	'Cửa' để Frontend nói chuyện với Backend. Mỗi cửa có địa chỉ và kiểu (GET để lấy, POST để tạo, PATCH để sửa, DELETE để xóa).	POST /auth/login
Authentication / Authorization	Xác thực = bạn là **ai**. Phân quyền = bạn được làm **gì**.	Đăng nhập là xác thực; chỉ giáo viên được điểm danh là phân quyền
RBAC	Phân quyền theo **vai trò**.	5 vai trò: ADMIN, MANAGER, TEACHER, MEDICAL_STAFF, PARENT
Token (JWT)	'Vé điện tử' hệ thống cấp sau khi đăng nhập; gửi kèm mỗi lần gọi API.	Hết hạn sau 15 phút
IDOR	Lỗi cho phép xem dữ liệu của người khác chỉ bằng cách đổi số id trên đường dẫn.	Phụ huynh A đổi /children/5 thành /children/6 và thấy con nhà B (**phải chặn**)
Audit log	Nhật ký ghi lại **ai đã làm gì, lúc nào**.	Ai sửa hồ sơ sức khỏe của bé An
Migration	File ghi lại mỗi thay đổi cấu trúc database, có phiên bản (giống Git cho database).	Migration thêm bảng attendance
Seed data	Dữ liệu mẫu nạp sẵn để chạy thử và demo.	5 tài khoản mẫu, 2 lớp, 10 bé
Docker	Đóng gói ứng dụng để chạy giống hệt nhau trên máy mọi người.	Lệnh docker compose up bật cả hệ thống
CI	'Người máy' tự chạy kiểm tra mỗi khi bạn đẩy code lên.	GitHub Actions chạy test khi mở Pull Request

2.5. Về Git và làm việc nhóm

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
Git / Repository	Công cụ lưu lịch sử code; repo là 'kho' chứa dự án.	Repo tên kidcare trên GitHub
Branch (nhánh)	Bản sao để làm tính năng mới mà không làm hỏng bản chính.	feature/auth-login
Commit	Một lần lưu thay đổi kèm lời nhắn.	feat(auth): add login endpoint
Pull Request (PR)	Đề nghị gộp nhánh của bạn vào nhánh chung; người khác xem xét (review) trước khi gộp.	B review PR đăng nhập của C
Conventional Commits	Quy ước viết lời nhắn commit: **loại(phạm vi): mô tả**.	fix(attendance): fix late status

2.6. Về kiểm thử

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
Unit test	Kiểm tra một hàm nhỏ chạy đúng.	Hàm tính trạng thái LATE
Integration test	Kiểm tra nhiều phần phối hợp đúng, thường có database thật.	Tạo trẻ và liên kết phụ huynh cùng lúc
API test	Gọi thử API và xem kết quả có đúng không.	Phụ huynh gọi API của giáo viên phải bị 403
E2E test	Mô phỏng người dùng bấm trên giao diện từ đầu đến cuối.	Đăng nhập → điểm danh → phụ huynh thấy

Thuật ngữ	Nghĩa dễ hiểu	Ví dụ trong KidCare
Regression	Chạy lại test **cũ** để chắc phần mới không làm hỏng phần cũ.	Sau khi làm Sức khỏe, chạy lại test Đăng nhập

3. Nhóm sẽ làm việc như thế nào?

3.1. Mô hình Incremental (giải thích bằng ví dụ)

Giả sử bạn xây một tòa nhà. Cách cũ: xây xong hết mới cho ai vào. Cách **Incremental**: xây xong tầng 1 thì cho vào dùng thử ngay, rồi xây tầng 2, tầng 3. Mỗi tầng xong là **dùng được**.

Với KidCare, mỗi 'tầng' là một Increment. Nhưng trước khi xây tầng 1, phải làm **móng** — đó là Increment 0: chốt yêu cầu tổng quát, thiết kế khung, dựng sẵn dự án trống. Móng vững thì các tầng sau mới không phải đập đi làm lại.

3.2. Lịch 12 tuần

Tuần	Làm gì	Xong thì có gì để cho xem?
1–2	**Increment 0 — Móng:** tài liệu yêu cầu, thiết kế khung, dựng dự án	Tài liệu + dự án trống chạy được bằng một lệnh
3–4	**Increment 1 — Đăng nhập và phân quyền**	Đăng nhập được 5 loại tài khoản
5–6	**Increment 2 — Quản lý trẻ, lớp, phụ huynh, giáo viên**	Tạo lớp, thêm trẻ, gắn phụ huynh
7–8	**Increment 3 — Điểm danh**	Cô điểm danh, phụ huynh xem con đã đến chưa
9	**Increment 4 — Sức khỏe**	Hồ sơ sức khỏe, tiêm chủng
10	**Increment 5 — Hoạt động và Thông báo**	Đăng hoạt động có ảnh, phụ huynh nhận thông báo
11	Kiểm thử tổng thể, đưa lên chạy (deploy)	Bản v1.0 chạy ổn định
12	Hoàn thiện tài liệu, chuẩn bị bảo vệ	Hồ sơ nộp bài, slide, kịch bản demo

> **Mẹo:** Thiếu thời gian thì cắt từ cuối lên: bỏ Reporting, rồi Pickup, rồi ảnh hoạt động. Còn **Increment 1–4 là phần lõi**, cố giữ. Xong Increment 3 là đã có một sản phẩm tối thiểu để nộp.

3.3. Vòng 8 bước lặp lại ở mỗi Increment

Increment nào cũng đi qua 8 bước này. Các phần 5–9 chỉ ghi những gì **riêng** của từng Increment.

Bước	Làm gì	Ví dụ với tính năng Điểm danh
1. Viết use case	Kể rõ từng bước người dùng làm	'Giáo viên mở lớp → bấm Check-in cho bé An → hệ thống ghi giờ đến'
2. Vẽ sơ đồ	Vẽ ra để cả nhóm hiểu giống nhau	Sequence Diagram: Giáo viên → Backend → Database
3. Thiết kế bảng dữ liệu	Quyết định lưu gì, ở bảng nào	Bảng attendance: trẻ nào, ngày nào, giờ đến, trạng thái
4. Viết danh sách API	Liệt kê các 'cửa' giữa Frontend và Backend	POST /attendance/check-in
5. Code Backend	Viết phần xử lý và lưu dữ liệu	Kiểm tra quyền, ghi vào database
6. Code Frontend	Làm giao diện gọi các API đó	Bảng điểm danh của lớp
7. Viết test	Kiểm tra chạy đúng và đúng quyền	Giáo viên lớp khác không được điểm danh
8. Demo và nhìn lại	Cả nhóm chạy thử, họp Retrospective, gắn phiên bản (tag)	Demo 10 phút, ghi biên bản

3.4. Vai trò 5 người

	Vai trò	Chịu trách nhiệm chính	Còn phải làm
A	Trưởng nhóm, người viết yêu cầu	Tài liệu yêu cầu, use case, danh sách việc, họp nhóm, báo cáo cuối	Review tài liệu của người khác; làm một phần code nhỏ

	Vai trò	Chịu trách nhiệm chính	Còn phải làm
B	Trưởng kỹ thuật, Backend	Thiết kế tổng thể, đăng nhập/phân quyền, review code backend	Hướng dẫn cả nhóm về kỹ thuật
C	Backend	Các module nghiệp vụ (trẻ, điểm danh, sức khỏe, hoạt động, thông báo)	Viết test cho phần mình làm
D	Frontend	Toàn bộ giao diện, bản phác giao diện	Cùng viết use case với A
E	Kiểm thử và vận hành	Kế hoạch test, test tự động, Docker, CI, đưa lên chạy	Viết dữ liệu mẫu, kiểm tra chéo phần người khác

Mẹo: Ai cũng phải **hiểu toàn hệ thống** vì lúc bảo vệ ai cũng có thể bị hỏi. Hãy luân phiên người viết tài liệu và người demo giữa các Increment.

3.5. Mỗi người nên học gì trước (tuần 0–2)

Người	Nên học trước
Cả nhóm	Git cơ bản (clone, branch, commit, push, pull request); TypeScript cơ bản; cách đọc tài liệu Swagger/OpenAPI
A	Cách viết yêu cầu (FR/NFR), use case, vẽ Use Case và Activity Diagram; Markdown
B, C	NestJS cơ bản; SQL và PostgreSQL; ý tưởng JWT, hash mật khẩu
D	React + TypeScript; gọi API bằng axios hoặc TanStack Query; Tailwind CSS
E	Docker và Docker Compose; Jest, Supertest; GitHub Actions; Playwright hoặc Cypress

Mẹo: Chưa biết công nghệ nào cũng không sao: làm theo cặp (một người rành kèm một người mới) và học qua chính các Pull Request.

4. Tuần 1 — Chuẩn bị nền tảng

Mục tiêu	Cả nhóm có chỗ chung để làm việc: kho code, bảng công việc, quy ước, lịch họp.
Ai làm	E dựng repo, A dựng bảng công việc; cả nhóm cùng thống nhất quy ước.

Các bước làm theo thứ tự

1. **Tạo repository** tên **kidcare** trên GitHub, mời cả nhóm. Tạo sẵn các thư mục theo Phụ lục A. Bật 'branch protection' cho nhánh **main** (không ai được push thẳng).
2. **Tạo nhánh chung** tên **develop**. Từ nay mỗi tính năng làm trên nhánh riêng **feature/...**, xong thì tạo Pull Request vào develop.
3. **Dựng bảng công việc** (GitHub Projects hoặc Trello) với các cột: Cần làm → Đang làm → Đang review → Xong. Mỗi thẻ ghi rõ ai làm và mã FR liên quan.
4. **Thống nhất quy ước commit** (Conventional Commits, xem phần 11) và mẫu Pull Request.
5. **Đặt lịch họp cố định:** 2–3 buổi 15 phút mỗi tuần (làm gì, vướng gì) và 1 buổi demo + retrospective cuối mỗi Increment. Ghi biên bản ngắn vào thư mục docs.
6. **Viết Project Charter** 1 trang: mục tiêu, phạm vi, ai làm gì, lịch, rủi ro chính.

Checklist

- ☐ Repo kidcare đã tạo, cả 5 người đều clone và push thử được.
- ☐ Nhánh main được bảo vệ; nhánh develop đã có.
- ☐ Bảng công việc đã dựng, có đủ 5 người.
- ☐ Cả nhóm đã đọc quy ước commit và làm thử một commit đúng chuẩn.
- ☐ Lịch họp cố định đã thống nhất và ghi vào tài liệu.
- ☐ Project Charter đã viết xong.
- ☐ Mỗi người đã biết mình cần học gì trước (mục 3.5) và bắt đầu học.

5. Increment 0 — Làm móng (Tuần 1–2)

Mục tiêu	Hiểu rõ bài toán, viết yêu cầu tổng quát, thiết kế khung, dựng dự án trống chạy được.
Nguyên tắc	Chỉ làm ở mức **tổng quát**. Chi tiết từng chức năng làm dần ở các Increment sau. Phần này quan trọng vì móng sai thì sửa rất tốn công.

Bước 1 — Viết Problem Statement (1 trang)

Trả lời 4 câu hỏi bằng lời của nhóm: **Ai** đang gặp vấn đề? Vấn đề **cụ thể** là gì? Phần mềm giúp **thế nào**? Phần mềm **không** làm những gì? Có thể lấy 7 vấn đề trong file KidCare_Project_Plan.md làm gốc: thông tin trẻ rời rạc, điểm danh thủ công, phụ huynh khó theo dõi, hồ sơ sức khỏe phân tán, hoạt động khó chia sẻ, thông báo không tập trung, đón trẻ chưa có xác thực.

Bước 2 — Xác định actor và stakeholder

Không có khách hàng thật, nên nhóm **giả lập**: với mỗi loại người dùng, viết 2–3 dòng về việc họ cần và điều họ lo ngại. Ghi rõ trong tài liệu 'đây là phân tích dựa trên bài toán giả định'.

Actor	Họ cần gì?	Họ lo gì?
Phụ huynh	Biết con đã đến trường chưa, tình trạng sức khỏe, hoạt động trong ngày	Người lạ xem được thông tin của con
Giáo viên	Điểm danh nhanh, ghi hoạt động, báo phụ huynh	Thao tác rườm rà, mất thời gian
Nhân viên y tế	Lưu hồ sơ sức khỏe đầy đủ, theo dõi tiêm chủng	Nhập sai hoặc mất dữ liệu
Quản lý	Nắm số liệu lớp, trẻ, giáo viên	Thiếu báo cáo tổng hợp
Admin	Quản lý tài khoản, phân quyền	Có người dùng vượt quyền

Bước 3 — Viết danh sách yêu cầu chức năng (FR)

Mẹo viết: **'[Actor] có thể [làm gì]'**. Mỗi FR một dòng, một mã, một độ ưu tiên. Mã theo dạng **FR-<MODULE>-số**. Ví dụ:

Mã	Yêu cầu	Ưu tiên	Increment
FR-AUTH-01	Người dùng đăng nhập bằng email và mật khẩu	Must	1
FR-AUTH-02	Mỗi vai trò chỉ dùng được chức năng của vai trò đó	Must	1
FR-CORE-01	Quản lý tạo lớp và xếp trẻ vào lớp	Must	2
FR-ATT-01	Giáo viên điểm danh trẻ (có mặt, vắng, muộn)	Must	3
FR-ATT-02	Phụ huynh xem con hôm nay đã đến trường chưa	Must	3
FR-HEA-01	Nhân viên y tế ghi chiều cao, cân nặng, dị ứng của trẻ	Must	4
FR-NOT-01	Khi trẻ được điểm danh, phụ huynh nhận thông báo	Should	5
FR-REP-01	Quản lý xem báo cáo chuyên cần theo lớp	Could	Tùy chọn

Bước 4 — Viết yêu cầu phi chức năng (NFR)

Chỉ viết những gì **kiểm tra được**. 'Phần mềm phải nhanh' là chưa đủ; 'API thông thường trả lời dưới 500 ms' mới đủ.

Mã	Yêu cầu	Cách kiểm tra
NFR-SEC-01	Mật khẩu lưu dạng mã hóa, không lưu nguyên văn	Xem bảng users trong database
NFR-SEC-02	Người dùng chỉ xem được dữ liệu của mình	Test IDOR: đổi id phải bị 403
NFR-PERF-01	API thông thường trả lời dưới 500 ms; mọi danh sách có phân trang	Đo với dữ liệu mẫu lớn
NFR-REL-01	Không mất dữ liệu điểm danh, có backup database	Thử khôi phục từ backup
NFR-MNT-01	Code chia module, có tài liệu API và test tự động	Xem cấu trúc repo, Swagger, CI

Bước 5 — Vẽ Use Case Diagram tổng thể

Hình người = actor, hình bầu dục = use case, đường nối = 'người này làm được việc này'. Vẽ bằng **draw.io** (miễn phí) hoặc PlantUML/Mermaid. Ở Increment 0 chỉ cần liệt kê use case ở mức cao (ví dụ 'Điểm danh', 'Quản lý hồ sơ sức khỏe'); từng use case sẽ được kể chi tiết ở Increment tương ứng.

Bước 6 — Vẽ ERD tổng thể

Chỉ cần tên bảng và quan hệ; cột chi tiết bổ sung sau. Hãy nắm 3 loại quan hệ chính:

Quan hệ	Nghĩa	Ví dụ
1 — N (một nhiều)	Một bên có nhiều bên kia	Một lớp có nhiều trẻ; mỗi trẻ thuộc một lớp
N — M (nhiều nhiều)	Hai bên đều có nhiều bên kia; cần một **bảng trung gian**	Một trẻ có nhiều phụ huynh (bố, mẹ, người giám hộ), một phụ huynh có nhiều con → bảng child_parents
1 — N theo thời gian	Mỗi ngày hoặc mỗi lần tạo một bản ghi mới	Một trẻ có nhiều bản ghi điểm danh, mỗi ngày một bản

Các bảng chính: users, parents, teachers, children, child_parents, classes, class_teachers, attendance, health_records, vaccinations, activities, activity_participants, notifications, audit_logs (và pickup_requests nếu làm Pickup).

Bước 7 — Chốt kiến trúc và quyết định kỹ thuật

Kiến trúc đơn giản gồm 4 khối: **Trình duyệt (React) → API (NestJS) → PostgreSQL**, thêm **Redis** cho các việc phụ như giới hạn số lần đăng nhập. Gọi là **Modular Monolith**: một ứng dụng duy nhất nhưng chia module gọn gàng. Ghi lại vài quyết định bằng ADR ngắn: chọn ORM nào (Prisma hay TypeORM), lưu token ra sao, lưu ảnh ở đâu.

> **Mẹo:** Quy tắc giữ code sạch: module này **không truy vấn thẳng bảng của module khác**; cần dữ liệu thì gọi qua 'cửa chính thức' (service) của module đó. Nhờ vậy sau này muốn tách thành hệ thống lớn hơn cũng dễ.

Bước 8 — Dựng dự án trống chạy được

- Cài Node.js và Docker trên máy mọi người.
- Backend: tạo dự án NestJS, tạo sẵn các module rỗng, thêm Swagger và một API kiểm tra sống **GET /health**.
- Frontend: tạo dự án React + TypeScript + Vite, thêm Router và Tailwind, làm layout khung.
- Docker Compose: chạy cùng lúc frontend, backend, PostgreSQL, Redis bằng một lệnh **docker compose up**.
- CI: thiết lập để mỗi Pull Request tự chạy lint, test, build.

Phân công

Người	Việc
A	Problem Statement, stakeholder, danh sách FR/NFR, Use Case Diagram, khởi tạo bảng truy vết
B	ERD tổng thể, kiến trúc, ADR, dựng NestJS + cấu trúc module
C	Phối hợp ERD, cấu hình database + migration + seed, Swagger
D	Bản phác giao diện khung (đăng nhập, layout, dashboard), dựng React + Vite
E	Repo, Docker Compose, CI, ESLint/Prettier, kế hoạch test, NFR

Checklist hoàn thành Increment 0

- ☐ Problem Statement, Stakeholder (giả lập) đã viết và cả nhóm đọc qua.
- ☐ Danh sách FR đủ các module, có mã, độ ưu tiên (MoSCoW) và Increment dự kiến.
- ☐ Danh sách NFR có tiêu chí kiểm tra được.
- ☐ Use Case Diagram tổng thể và danh sách use case đã có.
- ☐ ERD tổng thể đã vẽ, cả nhóm đã xem và đồng ý.

☐	Sơ đồ kiến trúc + ít nhất 3–4 ADR đã viết.
☐	Bản phác giao diện khung đã có.
☐	**docker compose up** chạy được cả hệ thống; mở được GET /health và trang Swagger.
☐	CI chạy xanh trên một Pull Request thử.
☐	**Cả nhóm đã chốt bằng văn bản: phần nào bắt buộc, phần nào tùy chọn.** (Nên xác nhận với giảng viên.)

6. Increment 1 — Đăng nhập và phân quyền (Tuần 3–4)

Sau Increment này, người dùng làm được gì?

- Đăng ký, đăng nhập, đăng xuất, đổi mật khẩu.
- Mỗi vai trò (5 vai trò) nhìn thấy màn hình và menu riêng.
- Admin quản lý được tài khoản và xem được nhật ký thao tác.

Bước 1 — Viết use case

Mỗi use case viết theo mẫu ở Phụ lục B (actor, điều kiện trước, các bước, trường hợp lỗi). Cần viết:

- Đăng ký, Đăng nhập, Đăng xuất, Đổi mật khẩu.
- Admin quản lý tài khoản (khóa, mở, đổi vai trò).
- Admin xem nhật ký (audit log).

Bước 2 — Vẽ sơ đồ

- **Activity Diagram:** luồng đăng nhập (nhập sai, bị khóa tạm, thành công).
- **Sequence Diagram:** đăng nhập → cấp token → gọi API có kiểm tra quyền.
- **Class Diagram:** User, Role, Token, AuditLog.

Bước 3 — Thiết kế bảng dữ liệu

Bảng	Lưu gì?
`users`	email, mật khẩu (đã mã hóa), họ tên, vai trò, trạng thái
`refresh_tokens`	'vé làm mới' để không phải đăng nhập lại liên tục
`audit_logs`	ai làm gì, lúc nào, giá trị cũ và mới, địa chỉ IP

Bước 4 — Viết danh sách API

Viết vào file openapi.yaml **trước khi code**, để Frontend và Backend làm song song mà vẫn khớp nhau.

API	Làm gì?
`POST /auth/register`	Đăng ký tài khoản
`POST /auth/login`	Đăng nhập, nhận token
`POST /auth/refresh`	Lấy token mới khi hết hạn
`POST /auth/logout`	Đăng xuất
`GET, PATCH /users/me`	Xem và sửa hồ sơ của mình
`GET /users, PATCH /users/:id/role`	Admin xem danh sách, đổi vai trò, khóa tài khoản
`GET /audit-logs`	Admin xem nhật ký

Bước 5 — Code Backend

- Mã hóa mật khẩu bằng thư viện có sẵn (bcrypt hoặc argon2); **không tự chế thuật toán**.
- Cấp token có hạn ngắn (khoảng 15 phút) kèm token làm mới.
- Viết 'người gác cổng' (Guard): mặc định **từ chối**, endpoint nào khai báo vai trò được phép mới cho qua.
- Kiểm tra dữ liệu đầu vào (email hợp lệ, mật khẩu đủ mạnh).
- Giới hạn số lần đăng nhập sai (ví dụ 5 lần mỗi phút).
- Ghi audit log khi đăng nhập, đổi vai trò, khóa tài khoản.
- Nạp dữ liệu mẫu: 1 admin, 1 manager, 2 giáo viên, 1 nhân viên y tế, 2 phụ huynh.

Bước 6 — Code Frontend

- Trang Đăng nhập, Đăng ký, Đổi mật khẩu; hiện lỗi rõ ràng.
- Lưu token an toàn và tự làm mới khi hết hạn.
- Chặn truy cập trang theo vai trò; menu và trang chủ khác nhau cho từng vai trò.
- Trang quản lý người dùng và trang xem nhật ký cho Admin.

Bước 7 — Viết test

- Test hàm mã hóa và kiểm tra token.
- Đăng ký → đăng nhập → làm mới → đăng xuất chạy đúng.
- **Test phân quyền:** với mỗi API, gọi bằng từng vai trò; vai trò không được phép phải nhận lỗi 403, chưa đăng nhập nhận 401.
- Thử đăng nhập sai nhiều lần; thử nhập ký tự lạ vào ô email.
- Test E2E: đăng nhập từng vai trò và thấy đúng trang chủ.

Bước 8 — Demo và nhìn lại

Cả nhóm chạy thử toàn bộ, chạy lại test của các Increment trước (regression), họp Retrospective 30 phút, cập nhật bảng truy vết, rồi gắn phiên bản (tag) lên Git.

> **Lưu ý:** Lỗi phổ biến nhất là **quên kiểm tra quyền** ở một API nào đó. Vì vậy hãy để mặc định là từ chối, rồi mới mở quyền cho từng nơi.

> **Lưu ý:** Không bao giờ commit mật khẩu thật hoặc khóa bí mật lên Git. Dùng file .env (đã đưa vào .gitignore).

Ai làm gì

Người	Việc
A	Viết use case đăng nhập/phân quyền, Activity Diagram, cập nhật bảng truy vết
B	Thiết kế cách cấp token, viết module auth và Guard phân quyền, review
C	Module users, audit log, dữ liệu mẫu
D	Các trang đăng nhập/đăng ký, chặn trang theo vai trò, trang quản lý user
E	Test case cho auth, test phân quyền, cập nhật CI

Checklist hoàn thành

- ☐ Đủ use case đăng nhập/phân quyền; Activity + Sequence Diagram đã vẽ.
- ☐ Bảng users, refresh_tokens, audit_logs khớp với migration thực tế.
- ☐ openapi.yaml có nhóm auth/users/audit-logs; Swagger hiển thị đúng.
- ☐ Đăng ký, đăng nhập, đăng xuất, làm mới token, đổi mật khẩu chạy đúng; mật khẩu đã mã hóa.
- ☐ 5 vai trò thấy đúng menu; gọi API trái quyền nhận 403.
- ☐ Giới hạn đăng nhập sai hoạt động.
- ☐ Audit log ghi được đăng nhập, đổi vai trò, khóa tài khoản.
- ☐ Có dữ liệu mẫu 5 vai trò, nạp được bằng một lệnh.
- ☐ Các test đạt; bảng truy vết đã cập nhật; demo nội bộ + retrospective xong; gắn tag **v0.1**.

7. Increment 2 — Quản lý trẻ, lớp, phụ huynh, giáo viên (Tuần 5–6)

Sau Increment này, người dùng làm được gì?

- Quản lý tạo lớp, thêm trẻ, gắn phụ huynh với trẻ, phân công giáo viên vào lớp.
- Phụ huynh chỉ thấy **con của mình**; giáo viên chỉ thấy **lớp mình phụ trách**.
- Danh sách có tìm kiếm, lọc và phân trang.

Bước 1 — Viết use case

Mỗi use case viết theo mẫu ở Phụ lục B (actor, điều kiện trước, các bước, trường hợp lỗi). Cần viết:

- Quản lý lớp; quản lý trẻ; quản lý phụ huynh; quản lý giáo viên.
- Gắn phụ huynh với trẻ; phân công giáo viên vào lớp.
- Phụ huynh xem thông tin con; giáo viên xem danh sách lớp.

Bước 2 — Vẽ sơ đồ

- **Activity Diagram:** tiếp nhận một bé mới (tạo hồ sơ → gắn phụ huynh → xếp lớp).
- **Sequence Diagram:** Phụ huynh xem con — hệ thống **kiểm tra bé đó có phải con của người này không** trước khi trả dữ liệu.
- **Class Diagram:** User, Parent, Teacher, Child, Class và các quan hệ N–M.

Bước 3 — Thiết kế bảng dữ liệu

Bảng	Lưu gì?
`parents, teachers`	thông tin thêm của phụ huynh và giáo viên, nối với bảng users
`classes`	tên lớp, năm học, trạng thái
`children`	họ tên, ngày sinh, giới tính, địa chỉ, ảnh đại diện, lớp, trạng thái
`child_parents`	bảng trung gian: trẻ nào — phụ huynh nào — quan hệ (bố, mẹ, người giám hộ)
`class_teachers`	bảng trung gian: lớp nào — giáo viên nào

Bước 4 — Viết danh sách API

Viết vào file openapi.yaml **trước khi code**, để Frontend và Backend làm song song mà vẫn khớp nhau.

API	Làm gì?
`GET, POST /classes`	Danh sách lớp, tạo lớp
`POST /classes/:id/teachers`	Phân công giáo viên vào lớp
`GET /classes/:id/children`	Danh sách trẻ trong lớp
`GET, POST /children`	Danh sách (tìm, lọc, phân trang), tạo trẻ
`GET, PATCH /children/:id`	Xem và sửa hồ sơ trẻ
`POST /children/:id/parents`	Gắn phụ huynh với trẻ
`GET /parents/me/children`	Phụ huynh xem các con của mình

Bước 5 — Code Backend

- **Kiểm tra quyền sở hữu:** viết một hàm dùng chung để kiểm tra 'trẻ này có phải con của người đang đăng nhập không' và 'lớp này có do giáo viên này phụ trách không'.
- Không xóa hẳn dữ liệu; đổi trạng thái thành INACTIVE (xóa mềm).
- Không cho xóa lớp còn trẻ đang học.
- Tạo trẻ và gắn phụ huynh trong **một transaction** (lỗi giữa chừng thì hoàn tác hết).

- Upload ảnh đại diện qua một lớp 'StorageService' (lưu vào thư mục trước, sau này dễ đổi sang dịch vụ lưu trữ khác); giới hạn loại và dung lượng file.
- Ghi audit log khi tạo, sửa, đổi lớp, vô hiệu hóa trẻ.
- Nạp dữ liệu mẫu: 2 lớp, 8–10 bé, mỗi bé 1–2 phụ huynh.

Bước 6 — Code Frontend

- Trang danh sách trẻ và lớp có ô tìm kiếm, bộ lọc, phân trang.
- Form tạo và sửa; trang chi tiết trẻ chia tab (Thông tin, Phụ huynh; các tab Điểm danh, Sức khỏe, Hoạt động để trống chờ Increment sau).
- Trang chủ Phụ huynh: thẻ 'Các con của tôi'; trang chủ Giáo viên: lớp phụ trách.
- Hỏi xác nhận trước khi làm việc nguy hiểm (vô hiệu hóa, đổi lớp).

Bước 7 — Viết test

- Test hàm kiểm tra quyền sở hữu.
- **Test IDOR:** phụ huynh A gọi API xem con của phụ huynh B phải nhận 403; giáo viên lớp khác không xem được lớp này.
- Test transaction: lỗi giữa chừng thì không có dữ liệu dở dang.
- Test E2E: Quản lý tạo lớp → tạo trẻ → gắn phụ huynh → phụ huynh đăng nhập thấy con.
- Chạy lại toàn bộ test Increment 1 (regression).

Bước 8 — Demo và nhìn lại

Cả nhóm chạy thử toàn bộ, chạy lại test của các Increment trước (regression), họp Retrospective 30 phút, cập nhật bảng truy vết, rồi gắn phiên bản (tag) lên Git.

> **Lưu ý: IDOR là lỗi nguy hiểm nhất của dự án này** vì dữ liệu là của trẻ em. Chỉ kiểm tra vai trò là chưa đủ; luôn kiểm tra thêm 'dữ liệu này có thuộc về người đang hỏi không'.

> **Mẹo:** Quan hệ N–M (trẻ–phụ huynh) là chỗ người mới hay vấp. Vẽ ra giấy: 1 bé có 2 phụ huynh, 1 phụ huynh có 2 bé, rồi mới thiết kế bảng child_parents.

Ai làm gì

Người	Việc
A	Use case quản lý lõi, Activity Diagram, cập nhật tài liệu, dữ liệu mẫu thực tế
B	Hàm kiểm tra quyền sở hữu, review, Sequence + Class Diagram
C	Các module classes, children, parents, teachers, StorageService
D	Toàn bộ trang danh sách/form/chi tiết; trang chủ Phụ huynh, Giáo viên
E	Test case, test IDOR, test E2E luồng tiếp nhận trẻ, regression

Checklist hoàn thành

- ☐ Đủ use case quản lý lõi; Activity, Sequence, Class Diagram đã có.
- ☐ 6 bảng khớp migration; openapi.yaml cập nhật.
- ☐ Thêm, sửa, tìm, lọc, phân trang lớp/trẻ/giáo viên/phụ huynh chạy đúng.
- ☐ Gắn phụ huynh–trẻ và phân công giáo viên–lớp hoạt động.
- ☐ Test IDOR đạt cho cả phụ huynh và giáo viên.
- ☐ Upload ảnh đại diện chạy, có giới hạn loại và dung lượng file.
- ☐ Audit log ghi được thao tác quan trọng trên hồ sơ trẻ.
- ☐ Đã có dữ liệu mẫu đủ để demo Increment 3.
- ☐ Test đạt, regression Increment 1 đạt; bảng truy vết cập nhật; demo + retrospective xong; gắn tag **v0.2**.

8. Increment 3 — Điểm danh (Tuần 7–8)

Sau Increment này, người dùng làm được gì?

- Giáo viên điểm danh cả lớp (có mặt, vắng, muộn, về sớm) và ghi giờ đến, giờ về.
- Phụ huynh xem 'hôm nay con đã đến trường chưa' và lịch sử điểm danh.
- Phụ huynh gửi đơn xin nghỉ; giáo viên duyệt hoặc từ chối.
- **Đây là tính năng chính để demo**.

Bước 1 — Viết use case

Mỗi use case viết theo mẫu ở Phụ lục B (actor, điều kiện trước, các bước, trường hợp lỗi). Cần viết:

- Điểm danh cả lớp; Check-in; Check-out; Sửa điểm danh trong ngày.
- Phụ huynh xem điểm danh hôm nay và lịch sử.
- Gửi đơn xin nghỉ; Duyệt hoặc từ chối đơn.

Bước 2 — Vẽ sơ đồ

- **Activity Diagram:** điểm danh buổi sáng (chưa điểm danh → điểm danh → tính trạng thái muộn → lưu).
- **Sequence Diagram:** Giáo viên check-in → Backend → Database → phát 'sự kiện' (dùng ở Increment 5 để gửi thông báo).
- **Class Diagram:** Attendance, LeaveRequest.

Bước 3 — Thiết kế bảng dữ liệu

Bảng	Lưu gì?
`attendance`	trẻ nào, ngày nào, giờ đến, giờ về, trạng thái, ai ghi; **mỗi trẻ chỉ một bản ghi mỗi ngày**
`leave_requests`	đơn xin nghỉ: trẻ nào, từ ngày nào đến ngày nào, lý do, trạng thái duyệt
`system_settings`	cấu hình như giờ vào lớp chuẩn (để xác định 'muộn')

Bước 4 — Viết danh sách API

Viết vào file openapi.yaml **trước khi code**, để Frontend và Backend làm song song mà vẫn khớp nhau.

API	Làm gì?
`GET /classes/:id/attendance?date=`	Bảng điểm danh của lớp theo ngày
`POST /attendance/check-in`	Ghi nhận trẻ đến
`POST /attendance/check-out`	Ghi nhận trẻ về
`PATCH /attendance/:id`	Sửa điểm danh (có ghi nhật ký)
`GET /children/:id/attendance/today`	Trạng thái hôm nay của một bé
`GET /children/:id/attendance`	Lịch sử điểm danh theo khoảng ngày
`POST /leave-requests`	Phụ huynh gửi đơn xin nghỉ
`PATCH /leave-requests/:id/approve`	Duyệt đơn (tương tự /reject để từ chối)

Bước 5 — Code Backend

- Quy tắc trạng thái: đến sau giờ chuẩn → MUỘN; về trước giờ chuẩn → VỀ SỚM; có đơn nghỉ được duyệt → VẮNG (có phép). **Viết thành một hàm riêng** để dễ test.
- Đặt ràng buộc **duy nhất (trẻ, ngày)** trong database để không thể có 2 bản ghi trùng.
- Chỉ giáo viên của lớp (hoặc Quản lý, Admin) mới điểm danh được; không điểm danh cho ngày trong tương lai.
- Sau khi lưu thành công, phát sự kiện **AttendanceRecorded** (module Điểm danh không gọi thẳng module Thông báo).

- Điểm danh cả lớp trong một transaction.
- Thêm chỉ mục (index) cho truy vấn 'hôm nay'; mọi danh sách có phân trang.

Bước 6 — Code Frontend

- Bảng điểm danh: chọn ngày, danh sách bé, nút đặt nhanh Có mặt / Vắng / Muộn, nút 'Lưu tất cả'.
- Trang phụ huynh 'Hôm nay con thế nào': trạng thái, giờ đến/về của từng con; lịch sử theo tháng.
- Form đơn xin nghỉ và màn hình duyệt đơn.
- Dùng tốt trên điện thoại (cô giáo thường thao tác bằng điện thoại).

Bước 7 — Viết test

- Test hàm tính trạng thái với các trường hợp sát biên: đúng giờ, muộn 1 phút, về sớm, có đơn nghỉ.
- Test không tạo được bản ghi trùng, kể cả khi hai yêu cầu gửi cùng lúc.
- Test quyền: giáo viên lớp khác bị 403; phụ huynh xem điểm danh con người khác bị 403.
- Test E2E: giáo viên điểm danh → phụ huynh mở 'Hôm nay' thấy đúng.
- Đo thời gian API 'hôm nay' với khoảng 10.000 bản ghi mẫu (mục tiêu dưới 500 ms).
- Chạy lại test Increment 1–2.

Bước 8 — Demo và nhìn lại

Cả nhóm chạy thử toàn bộ, chạy lại test của các Increment trước (regression), họp Retrospective 30 phút, cập nhật bảng truy vết, rồi gắn phiên bản (tag) lên Git.

> **Mẹo:** Nếu nhóm chỉ kịp làm được tới đây, bạn vẫn có một sản phẩm hoàn chỉnh và có điểm nhấn. Hãy làm phần này thật chắc.

> **Lưu ý:** Hai giáo viên bấm điểm danh cùng một bé cùng lúc là chuyện có thật. Ràng buộc 'duy nhất (trẻ, ngày)' là lớp bảo vệ cuối cùng; đừng chỉ kiểm tra ở code.

Ai làm gì

Người	Việc
A	Use case điểm danh, quy tắc trạng thái viết thành văn bản, tài liệu, bảng truy vết
B	Thiết kế sự kiện + xử lý điểm danh cùng lúc, review, Sequence Diagram
C	Module điểm danh, đơn xin nghỉ, phát sự kiện, cấu hình giờ chuẩn
D	Bảng điểm danh, trang phụ huynh, form và duyệt đơn nghỉ
E	Test trạng thái, test đồng thời, đo hiệu năng, dữ liệu mẫu lớn, regression

Checklist hoàn thành

- ☐ Đủ use case điểm danh và đơn nghỉ; các sơ đồ đã vẽ.
- ☐ 3 bảng khớp migration; ràng buộc duy nhất và index đã tạo; openapi.yaml cập nhật.
- ☐ Điểm danh, check-in/out, sửa điểm danh chạy đúng; trạng thái MUỘN/VỀ SỚM tính đúng.
- ☐ Không tạo được bản ghi trùng (trẻ, ngày), kể cả khi gửi cùng lúc.
- ☐ Phụ huynh xem được 'Hôm nay' và lịch sử; chỉ thấy con mình.
- ☐ Đơn xin nghỉ: gửi → duyệt/từ chối → phản ánh vào điểm danh.
- ☐ Sự kiện AttendanceRecorded được phát ra (chứng minh bằng test hoặc log).
- ☐ API 'hôm nay' đạt mục tiêu tốc độ.
- ☐ Test đạt, regression Inc 1–2 đạt; demo + retrospective xong; gắn tag **v0.3**. **Mốc đủ điều kiện nộp bản tối thiểu.**

9. Increment 4 — Sức khỏe (Tuần 9)

Sau Increment này, người dùng làm được gì?

- Nhân viên y tế ghi chiều cao, cân nặng, nhiệt độ, dị ứng, bệnh nền, tiêm chủng và các lần khám.
- Phụ huynh xem được tóm tắt sức khỏe của con.
- Giáo viên chỉ thấy phần cần thiết (ví dụ cảnh báo dị ứng), **không thấy** chẩn đoán chi tiết.

Bước 1 — Viết use case

Mỗi use case viết theo mẫu ở Phụ lục B (actor, điều kiện trước, các bước, trường hợp lỗi). Cần viết:

- Tạo và sửa hồ sơ sức khỏe; ghi số đo; quản lý dị ứng và bệnh nền.
- Quản lý tiêm chủng; ghi lần khám.
- Phụ huynh xem tóm tắt; giáo viên xem cảnh báo dị ứng.

Bước 2 — Vẽ sơ đồ

- **Activity Diagram:** khám định kỳ và cập nhật hồ sơ.
- **Sequence Diagram:** Nhân viên y tế cập nhật → ghi audit log (giá trị cũ, mới).
- **Bảng quyền theo từng trường dữ liệu:** vai trò nào xem/sửa trường nào (ví dụ giáo viên xem dị ứng nhưng không xem chẩn đoán).

Bước 3 — Thiết kế bảng dữ liệu

Bảng	Lưu gì?
`health_records`	chiều cao, cân nặng, nhiệt độ, dị ứng, bệnh nền, ghi chú, người ghi, thời điểm
`vaccinations`	tên vắc-xin, mũi thứ mấy, ngày tiêm, ngày hẹn mũi tiếp theo
`medical_visits`	ngày khám, triệu chứng, chẩn đoán, cách xử lý, người khám

Bước 4 — Viết danh sách API

Viết vào file openapi.yaml **trước khi code**, để Frontend và Backend làm song song mà vẫn khớp nhau.

API	Làm gì?
`GET, POST /children/:id/health-records`	Xem lịch sử và thêm số đo
`PATCH /health-records/:id`	Sửa bản ghi (có ghi nhật ký)
`GET /children/:id/health-summary`	Tóm tắt sức khỏe, **đã lọc theo vai trò**
`GET, POST /children/:id/vaccinations`	Quản lý tiêm chủng
`GET, POST /children/:id/medical-visits`	Các lần khám
`GET /classes/:id/health-alerts`	Danh sách bé có dị ứng/lưu ý trong lớp

Bước 5 — Code Backend

- **Lọc dữ liệu theo vai trò:** viết một chỗ riêng quyết định vai trò nào thấy trường nào, thay vì rải điều kiện khắp nơi.
- Kiểm tra giá trị hợp lý (chiều cao, cân nặng, nhiệt độ không thể âm hoặc quá vô lý).
- Ghi audit log cho **mọi** thao tác sửa dữ liệu sức khỏe (giá trị cũ và mới).
- Endpoint liệt kê các mũi tiêm sắp đến hạn (tự động hóa nhắc lịch để sau).
- Không đưa chi tiết sức khỏe vào file log.

Bước 6 — Code Frontend

- Tab 'Sức khỏe' trong trang chi tiết trẻ: số đo, dị ứng và bệnh nền, tiêm chủng, lần khám.
- Form nhập chỉ hiện cho nhân viên y tế.
- Biểu đồ chiều cao, cân nặng theo thời gian (nếu còn thời gian).

- Biểu tượng cảnh báo dị ứng trong danh sách lớp của giáo viên.

Bước 7 — Viết test

- Test kiểm tra giá trị hợp lý.
- Test lọc theo vai trò: phụ huynh không thấy phần chẩn đoán chi tiết; giáo viên chỉ thấy cảnh báo.
- Test sửa hồ sơ có tạo audit log đúng giá trị cũ và mới.
- Test E2E: nhân viên y tế nhập tiêm chủng → phụ huynh thấy trong hồ sơ con.
- Chạy lại test Increment 1–3.

Bước 8 — Demo và nhìn lại

Cả nhóm chạy thử toàn bộ, chạy lại test của các Increment trước (regression), họp Retrospective 30 phút, cập nhật bảng truy vết, rồi gắn phiên bản (tag) lên Git.

> **Lưu ý:** Dữ liệu sức khỏe là dữ liệu **nhạy cảm**. Hai điều phải làm: chỉ trả về đúng phần người hỏi được xem, và ghi lại mọi thay đổi.

Ai làm gì

Người	Việc
A	Use case sức khỏe, bảng quyền theo trường, tài liệu, bảng truy vết
B	Thiết kế phần lọc theo vai trò và audit, review
C	Module sức khỏe (3 thực thể), API cảnh báo dị ứng
D	Tab sức khỏe, form nhập, biểu đồ, cảnh báo dị ứng
E	Test quyền theo trường, test audit, E2E, regression

Checklist hoàn thành

- ☐ Đủ use case sức khỏe; các sơ đồ đã vẽ.
- ☐ Bảng quyền theo trường đã viết thành tài liệu **và** đã cài đặt trong code.
- ☐ 3 bảng khớp migration; openapi.yaml cập nhật.
- ☐ Tạo, sửa, xem hồ sơ sức khỏe, tiêm chủng, lần khám đúng quyền.
- ☐ Phụ huynh và giáo viên chỉ thấy phần được phép.
- ☐ Mọi thay đổi dữ liệu sức khỏe đều có audit log.
- ☐ Giao diện tab sức khỏe và cảnh báo dị ứng hoạt động.
- ☐ Test đạt, regression Inc 1–3 đạt; bảng truy vết cập nhật; demo + retrospective xong; gắn tag **v0.4**.

10. Increment 5 — Hoạt động và Thông báo (Tuần 10)

Sau Increment này, người dùng làm được gì?

- Giáo viên tạo hoạt động (vẽ tranh, đọc sách, thể dục...), chọn bé tham gia, đăng ảnh.
- Phụ huynh xem hoạt động của con.
- Khi có sự kiện (điểm danh, hoạt động, cập nhật sức khỏe), **hệ thống tự gửi thông báo** đến đúng phụ huynh.
- Phụ huynh xem thông báo, đánh dấu đã đọc; có chuông báo số chưa đọc.

Bước 1 — Viết use case

Mỗi use case viết theo mẫu ở Phụ lục B (actor, điều kiện trước, các bước, trường hợp lỗi). Cần viết:

- Tạo hoạt động; ghi nhận trẻ tham gia; tải ảnh hoạt động.
- Phụ huynh xem hoạt động của con.
- Hệ thống tự tạo thông báo từ sự kiện; xem danh sách thông báo; đánh dấu đã đọc; giáo viên gửi thông báo thủ công.

Bước 2 — Vẽ sơ đồ

- **Sequence Diagram quan trọng nhất dự án:** Giáo viên check-in → sự kiện AttendanceRecorded → module Thông báo lắng nghe → tạo thông báo → phụ huynh thấy chuông báo.
- **Activity Diagram:** tạo hoạt động có ảnh và ghi nhận tham gia.
- **Class Diagram:** Activity, ActivityImage, Notification, NotificationChannel.

Bước 3 — Thiết kế bảng dữ liệu

Bảng	Lưu gì?
`activities`	tiêu đề, mô tả, ngày, lớp, người tạo
`activity_participants`	bảng trung gian: hoạt động nào — bé nào — tham gia thế nào
`activity_images`	chỉ lưu **thông tin** ảnh (khóa file, đường dẫn); **không** lưu ảnh trong database
`notifications`	gửi cho ai, loại gì, tiêu đề, nội dung, đã đọc chưa, thời gian

Bước 4 — Viết danh sách API

Viết vào file openapi.yaml **trước khi code**, để Frontend và Backend làm song song mà vẫn khớp nhau.

API	Làm gì?
`GET, POST /activities`	Danh sách hoạt động, tạo hoạt động
`POST /activities/:id/participants`	Ghi nhận bé tham gia
`POST /activities/:id/images`	Tải ảnh lên
`GET /children/:id/activities`	Hoạt động của một bé
`GET /notifications`	Thông báo của tôi (phân trang)
`GET /notifications/unread-count`	Số thông báo chưa đọc
`PATCH /notifications/:id/read`	Đánh dấu đã đọc
`POST /notifications`	Giáo viên gửi thông báo cho lớp

Bước 5 — Code Backend

- Module Thông báo **lắng nghe sự kiện** từ các module khác (điểm danh, sức khỏe, hoạt động) chứ không để các module đó gọi thẳng vào.
- Xác định người nhận: từ bé → các phụ huynh của bé (bảng child_parents).
- Làm 'kênh gửi' (NotificationChannel) là một giao diện chung; hiện chỉ có kênh IN_APP, sau này thêm Email/SMS/Push mà không phải sửa nhiều.

- Nội dung thông báo **không chứa chi tiết nhạy cảm** (chỉ nói loại sự kiện và tham chiếu).
- Upload ảnh: chỉ nhận jpg/png/webp, giới hạn dung lượng, đổi tên ngẫu nhiên, lưu qua StorageService.
- Chỉ giáo viên của lớp mới tạo hoạt động cho lớp đó.

Bước 6 — Code Frontend

- Trang hoạt động: danh sách, chi tiết, thư viện ảnh, form tạo (chọn bé, tải ảnh).
- Chuông thông báo trên thanh menu: số chưa đọc, danh sách, đánh dấu đã đọc.
- Cập nhật thông báo bằng cách hỏi lại server mỗi 30 giây (polling) là đủ.
- Trang giáo viên gửi thông báo thủ công.

Bước 7 — Viết test

- Test xác định đúng người nhận từ mã bé.
- **Test luồng sự kiện:** check-in một bé → có thông báo cho **đúng** phụ huynh của bé đó, không phải người khác.
- Test người dùng chỉ đọc được thông báo của chính mình.
- Test upload: file sai loại hoặc quá lớn bị từ chối.
- Test E2E xuyên suốt: giáo viên điểm danh → chuông của phụ huynh hiện thông báo → đọc → số chưa đọc giảm.
- Chạy lại test Increment 1–4.

Bước 8 — Demo và nhìn lại

Cả nhóm chạy thử toàn bộ, chạy lại test của các Increment trước (regression), họp Retrospective 30 phút, cập nhật bảng truy vết, rồi gắn phiên bản (tag) lên Git.

> **Mẹo:** Ý tưởng 'phát sự kiện, ai quan tâm thì lắng nghe' giống chuông cửa: người bấm chuông không cần biết ai sẽ ra mở. Đây cũng là nền để sau này học Message Queue ở môn Thiết kế hệ thống.

Ai làm gì

Người	Việc
A	Use case hoạt động/thông báo, Sequence xuyên module, tài liệu, bảng truy vết
B	Thiết kế cơ chế sự kiện và kênh thông báo, review
C	Module hoạt động, thông báo, phần lắng nghe sự kiện, upload ảnh
D	Trang hoạt động, chuông thông báo, gửi thông báo thủ công
E	Test luồng sự kiện → thông báo, E2E xuyên suốt, test upload, regression toàn bộ

Checklist hoàn thành

- ☐ Đủ use case hoạt động và thông báo; Sequence Diagram xuyên module đã có.
- ☐ 4 bảng khớp migration; index bảng thông báo đã tạo; openapi.yaml cập nhật.
- ☐ Tạo hoạt động, ghi nhận bé tham gia, tải ảnh chạy đúng; ảnh không lưu trong PostgreSQL.
- ☐ Check-in tự sinh thông báo IN_APP cho đúng phụ huynh; cập nhật sức khỏe/hoạt động cũng sinh thông báo.
- ☐ Chuông thông báo, số chưa đọc, đánh dấu đã đọc hoạt động.
- ☐ Giáo viên gửi được thông báo thủ công.
- ☐ NotificationChannel và StorageService là giao diện chung; đã ghi cách mở rộng trong tài liệu.
- ☐ Test đạt, regression Inc 1–4 đạt; demo + retrospective xong; gắn tag **v0.5**.

11. Hoàn thiện và bảo vệ (Tuần 11–12)

Mục tiêu	Biến bản v0.5 thành bản v1.0: kiểm thử tổng thể, đưa lên chạy, hoàn thiện tài liệu, chuẩn bị bảo vệ.
Lưu ý	Tuần 12 **không thêm tính năng mới**. Nếu tuần 11 còn dư thời gian, làm thêm Pickup hoặc Reporting (mục 11.4).

11.1. Kiểm thử tổng thể (Tuần 11)

- Chạy toàn bộ test (unit, integration, API, E2E); sửa hết lỗi nghiêm trọng (Critical/Major).
- **Kiểm tra bảo mật:** thử IDOR trên **mọi** API có :id; thử đăng nhập sai liên tục; thử nhập ký tự lạ; kiểm tra token hết hạn hoặc giả mạo; đảm bảo không lộ mật khẩu hoặc thông tin lỗi nội bộ.
- **Đối chiếu NFR:** tốc độ API, phân trang, backup và khôi phục database thử được.
- **Kiểm thử chéo:** mỗi người chạy kịch bản của một vai trò mà mình **không** phải người làm; nhờ 1–2 bạn ngoài nhóm dùng thử và góp ý.
- Viết Test Report: số test case, tỷ lệ đạt, danh sách lỗi và tình trạng, ảnh chụp minh chứng.

11.2. Đưa phần mềm lên chạy — deploy (Tuần 11)

- Viết Dockerfile bản chạy thật cho backend và frontend; cấu hình Nginx.
- Đưa bí mật (mật khẩu database, khóa token) vào biến môi trường; hoàn thiện file .env.example.
- Deploy lên VPS hoặc dịch vụ như Render, Railway; bật HTTPS; kiểm tra /health.
- Viết script backup PostgreSQL và hướng dẫn khôi phục.
- **Luôn chuẩn bị bản dự phòng chạy trên máy cá nhân** (docker compose up + dữ liệu mẫu) phòng khi mạng hoặc deploy hỏng lúc demo.

11.3. Hoàn thiện tài liệu (Tuần 11–12)

Tài liệu	Gộp từ	Người phụ trách
Đặc tả yêu cầu (SRS)	Problem Statement, Stakeholder, FR, NFR, các use case	A
Tài liệu thiết kế	Các sơ đồ UML, ERD, kiến trúc, ADR, openapi.yaml, bản phác giao diện	B
Kế hoạch và Báo cáo kiểm thử	Test case các Increment, kết quả test, danh sách lỗi	E
Hướng dẫn cài đặt và sử dụng	README, hướng dẫn theo từng vai trò kèm ảnh chụp màn hình	D và E
Báo cáo cuối kỳ	Quy trình đã làm (kèm bằng chứng: commit, PR, biên bản), bài học, hướng phát triển	A và cả nhóm
Bảng truy vết hoàn chỉnh	FR → Use case → thiết kế → API → test case	A và E

11.4. Phần tùy chọn: Pickup và Reporting

- **Pickup (người đón trẻ):** Phụ huynh đăng ký người đón → giáo viên xác minh → bé được bàn giao → ghi nhật ký và thông báo. Trạng thái: PENDING, APPROVED, REJECTED, COMPLETED.
- **Reporting (báo cáo):** số bé đi học trong ngày, tỷ lệ chuyên cần, số bé đi muộn; chỉ Quản lý và Admin xem được.
- **Nếu không kịp code:** vẫn viết use case, sơ đồ, bảng dữ liệu, API cho hai phần này trong tài liệu và ghi rõ 'chưa cài đặt'. Vẫn được điểm cho phần thiết kế.

11.5. Chuẩn bị bảo vệ (Tuần 12)

- Slide 12–15 trang theo mạch: bài toán → quy trình Incremental → kiến trúc → demo → kiểm thử → bài học → hướng phát triển.
- **Kịch bản demo** theo đúng thứ tự: Quản lý tạo lớp và bé → Giáo viên điểm danh → Phụ huynh nhận thông báo và xem hồ sơ con → Nhân viên y tế cập nhật sức khỏe. Chuẩn bị sẵn tài khoản và dữ liệu.
- Quay video demo dự phòng.

- Mỗi người nắm chắc phần mình làm **và** luồng xuyên suốt (điểm danh → sự kiện → thông báo). Tập dượt ít nhất 2 lần, tự đặt câu hỏi cho nhau.

Checklist giai đoạn cuối

☐	Chạy lại toàn bộ test đạt; không còn lỗi nghiêm trọng.
☐	Đã kiểm tra bảo mật, gồm IDOR trên mọi API có :id.
☐	Đã đối chiếu NFR (tốc độ, phân trang, backup) và ghi kết quả.
☐	Bản deploy truy cập được qua HTTPS; đã thử backup và khôi phục database.
☐	CI chạy xanh trên main; đã gắn tag **v1.0**.
☐	SRS, Tài liệu thiết kế, Báo cáo kiểm thử, Hướng dẫn cài đặt/sử dụng, Báo cáo cuối đã xong và khớp nhau.
☐	Bảng truy vết đầy đủ: mọi FR bắt buộc đều có use case, thiết kế, API, test case.
☐	openapi.yaml khớp API thực tế; README ghi đúng cách chạy.
☐	Pickup/Reporting: đã làm, hoặc đã có tài liệu thiết kế ghi 'chưa cài đặt'.
☐	Slide, kịch bản demo, video demo và bản chạy dự phòng đã sẵn sàng.
☐	Đã tập dượt bảo vệ ít nhất 2 lần.

12. Quy tắc làm việc nhóm

12.1. Git — quy trình một tính năng

Đây là các bước bạn sẽ lặp lại hàng chục lần. Ví dụ làm tính năng đăng nhập:

```
git checkout develop
git pull
git checkout -b feature/auth-login
... viet code ...
git add .
git commit -m "feat(auth): add login endpoint"
git push -u origin feature/auth-login
```

1. Trên GitHub, mở **Pull Request** từ nhánh feature vào **develop**; điền mẫu mô tả (làm gì, use case nào, cách kiểm tra).
2. Ít nhất **một người khác** xem (review) và bấm duyệt; CI phải xanh. Bạn **không tự duyệt** PR của mình.
3. Duyệt xong thì merge. Cuối mỗi Increment, merge develop vào main và gắn tag phiên bản (v0.1, v0.2...).

> **Mẹo:** Không ai push thẳng vào main hoặc develop. Pull Request nhỏ (dưới khoảng 400 dòng) thì review nhanh và ít lỗi hơn.

12.2. Viết lời nhắn commit (Conventional Commits)

Cấu trúc: **loại(phạm vi): mô tả ngắn**. Viết ở thể mệnh lệnh, chữ thường, không dấu chấm cuối.

Loại	Dùng khi	Ví dụ
feat	Thêm tính năng mới	`feat(attendance): add check-in endpoint`
fix	Sửa lỗi	`fix(attendance): fix late status calculation`
docs	Chỉ sửa tài liệu	`docs(requirements): add UC-ATT-02`
test	Thêm hoặc sửa test	`test(children): add IDOR test`
refactor	Sắp xếp lại code, không đổi hành vi	`refactor(notifications): extract channel interface`
chore / ci	Việc vặt, cấu hình, CI	`ci: add lint step`

12.3. Khi nào một việc được coi là 'xong' (Definition of Done)

- ☐ Có mã FR hoặc use case liên quan, và đã viết đặc tả **trước** khi code.
- ☐ Đã qua Pull Request và có ít nhất một người review.
- ☐ Có test tự động cho logic chính **và** cho quyền truy cập; CI xanh.
- ☐ Đã cập nhật openapi.yaml; thay đổi database có migration đi kèm.
- ☐ Không có mật khẩu hay khóa bí mật thật trong repo.
- ☐ Đã cập nhật bảng truy vết và tài liệu liên quan.
- ☐ Chạy được trên nhánh develop bằng docker compose up.

12.4. Họp và ghi chép

- **Họp ngắn 15 phút, 2–3 lần mỗi tuần:** mỗi người trả lời 3 câu — Tôi đã làm gì? Tôi sẽ làm gì? Tôi đang vướng gì?
- **Cuối mỗi Increment:** demo nội bộ, chạy lại test, họp Retrospective (giữ gì, bỏ gì, thử gì), ghi biên bản.
- Mọi thay đổi phạm vi hoặc yêu cầu phải ghi lại và được cả nhóm đồng ý; A giữ bản 'chuẩn' của danh sách yêu cầu.
- Lịch sử commit, Pull Request và biên bản họp chính là **bằng chứng** nhóm đã làm theo mô hình Incremental. Đừng xóa lịch sử.

12.5. Quy tắc code cần nhớ

- Mỗi module một thư mục; module này không truy vấn thẳng bảng của module khác.

- Mọi API mặc định **từ chối** nếu chưa khai báo ai được dùng; luôn kiểm tra cả quyền sở hữu dữ liệu, không chỉ vai trò.
- Mọi danh sách phải phân trang; không log dữ liệu nhạy cảm; không trả thừa thông tin của trẻ.
- Thay đổi cấu trúc database luôn qua migration, không sửa tay trong database.

13. Nếu gặp khó khăn

Tình huống	Nên làm gì
Trễ tiến độ	Nói ngay trong buổi họp gần nhất. Cắt phạm vi theo thứ tự: Reporting → Pickup → ảnh hoạt động → biểu đồ sức khỏe. Không cắt tài liệu và test của phần bắt buộc.
Không biết công nghệ	Làm theo cặp với người rành hơn; đọc tài liệu chính thức của công cụ; hỏi trong nhóm trước khi kẹt quá 2 giờ.
Code của hai người bị xung đột (conflict) khi merge	Đừng hoảng. Cập nhật nhánh của bạn từ develop thường xuyên (mỗi ngày), làm PR nhỏ, và nhờ người viết đoạn kia cùng giải quyết.
Thiết kế ban đầu sai	Nếu chỉ sai một bảng nhỏ: sửa bằng migration mới. Nếu ảnh hưởng lớn: họp cả nhóm, ghi ADR về thay đổi trước khi làm.
Tài liệu và code không khớp	Cuối mỗi Increment, đối chiếu bảng truy vết; sửa tài liệu **hoặc** code cho khớp nhau.
Một thành viên vắng hoặc ít tham gia	Nói chuyện sớm và thẳng thắn; giao lại việc nhỏ, rõ ràng; mỗi module luôn có người phụ trách chính và người phụ (review).
Lỗi khi demo	Dùng bản chạy dự phòng và video đã quay sẵn; tập dượt trước.

Phụ lục A — Cấu trúc thư mục

Bản đầy đủ nằm trong README.md. Đây là bản rút gọn để bạn nắm ý nghĩa từng phần.

Thư mục	Chứa gì	Ai chủ yếu làm
`docs/00-management`	Project Charter, biên bản họp, retrospective, rủi ro	A
`docs/01-requirements`	Problem Statement, Stakeholder, FR, NFR, use case, bảng truy vết	A
`docs/02-design`	Các sơ đồ UML, ERD, kiến trúc, openapi.yaml, bản phác giao diện, ADR	B, D
`docs/03-testing`	Kế hoạch test, test case, báo cáo test	E
`docs/04-deployment`	Hướng dẫn cài đặt và vận hành	E
`docs/05-user-guide`	Hướng dẫn sử dụng theo vai trò	D, E
`docs/06-final-report`	Báo cáo cuối, slide, kịch bản demo	A
`backend/src`	Code backend, mỗi module một thư mục con (auth, users, children, classes, attendance, health, activities, notifications...)	B, C
`backend/database`	Migration (lịch sử thay đổi database) và seed (dữ liệu mẫu)	B, C
`backend/test`	Test tích hợp và E2E	E, C
`frontend/src`	Code giao diện: api, components, layouts, routes, pages, hooks...	D
`deploy`	Nginx, Docker Compose bản chạy thật, script backup	E
`.github`	CI (workflow) và mẫu Pull Request	E

Phụ lục B — Mẫu tài liệu

B.1. Mẫu đặc tả Use Case

Mục	Nội dung (ví dụ)
Mã và tên	UC-ATT-02 — Check-in cho trẻ
Actor	Giáo viên (chính); Quản lý (phụ)
Mô tả	Giáo viên ghi nhận trẻ đã đến lớp trong ngày
Điều kiện trước	Giáo viên đã đăng nhập, được phân công vào lớp của bé; bé chưa check-in hôm nay
Các bước chính	1. Giáo viên mở bảng điểm danh của lớp. 2. Chọn bé, bấm Check-in. 3. Hệ thống ghi giờ và tính trạng thái (có mặt hoặc muộn). 4. Hệ thống lưu và phát sự kiện. 5. Hiển thị kết quả.
Trường hợp khác	3a. Bé đã có bản ghi hôm nay → báo lỗi và gợi ý sửa. 3b. Có đơn nghỉ được duyệt → cảnh báo trước khi ghi.
Kết quả	Có đúng một bản ghi điểm danh cho (bé, ngày); sự kiện đã phát; audit log đã ghi
Yêu cầu liên quan	FR-ATT-01, NFR-REL-01

B.2. Mẫu bảng truy vết

Đọc từ trái sang phải: yêu cầu nào → use case nào → thiết kế ở đâu → API nào → test nào → đã xong chưa. Nhờ đó chứng minh **không sót yêu cầu** nào và **không có code thừa** không ai yêu cầu.

FR	Use case	Thiết kế	API	Test case	Trạng thái
FR-AUTH-01	UC-AUTH-02	SEQ-AUTH-01	POST /auth/login	TC-AUTH-01 → 05	Xong
FR-ATT-01	UC-ATT-02	SEQ-ATT-01	POST /attendance/check-in	TC-ATT-04 → 09	Đang làm

FR	Use case	Thiết kế	API	Test case	Trạng thái
FR-HEA-01	UC-HEA-02	CLS-HEA-01	POST /children/:id/health-records	TC-HEA-01 → 04	Chưa

Quy ước mã: FR (yêu cầu), UC (use case), SEQ/ACT/CLS (sơ đồ Sequence, Activity, Class), TC (test case), theo sau là tên module và số thứ tự.

Phụ lục C — Checklist nộp bài

- ☐ Mã nguồn sạch, README hướng dẫn chạy, .env.example, không có bí mật thật.
- ☐ Lịch sử commit, Pull Request, tag phiên bản v0.1 → v1.0 còn nguyên.
- ☐ SRS, Tài liệu thiết kế (đủ 4 loại sơ đồ UML + ERD + kiến trúc + API), Kế hoạch và Báo cáo kiểm thử.
- ☐ Báo cáo cuối kỳ mô tả quy trình Incremental đã áp dụng, kèm bằng chứng (biên bản, retrospective, bảng công việc).
- ☐ Hướng dẫn cài đặt và sử dụng; bản deploy hoặc bản chạy bằng Docker.
- ☐ Slide, kịch bản demo, video demo dự phòng.
- ☐ Bảng phân công và mức đóng góp của từng thành viên.
- ☐ Bảng truy vết hoàn chỉnh.